

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

#	Scan ID 19	✦	Date August 14, 2026
📦	Organization RigoBlock	📦	Repository v3-contracts
📄	Branch feat/hyperliquid	📄	Commit Hash 656099d7...2101fa4d

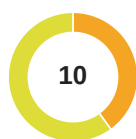
Contracts in scope

contracts/protocol/libraries/HyperliquidLib.sol contracts/protocol/extensions/adapters/AHyperliquid.sol
contracts/protocol/extensions/adapters/interfaces/IAHyperliquid.sol

Code Statistics

🐛	Findings 10	📄	Contracts Scanned 3	📄	Lines of Code 267
---	----------------	---	------------------------	---	----------------------

Findings Summary



Total Findings

 High Risk (0) Medium Risk (4) Low Risk (6) Info (0) Best Practices (0)

Code Summary

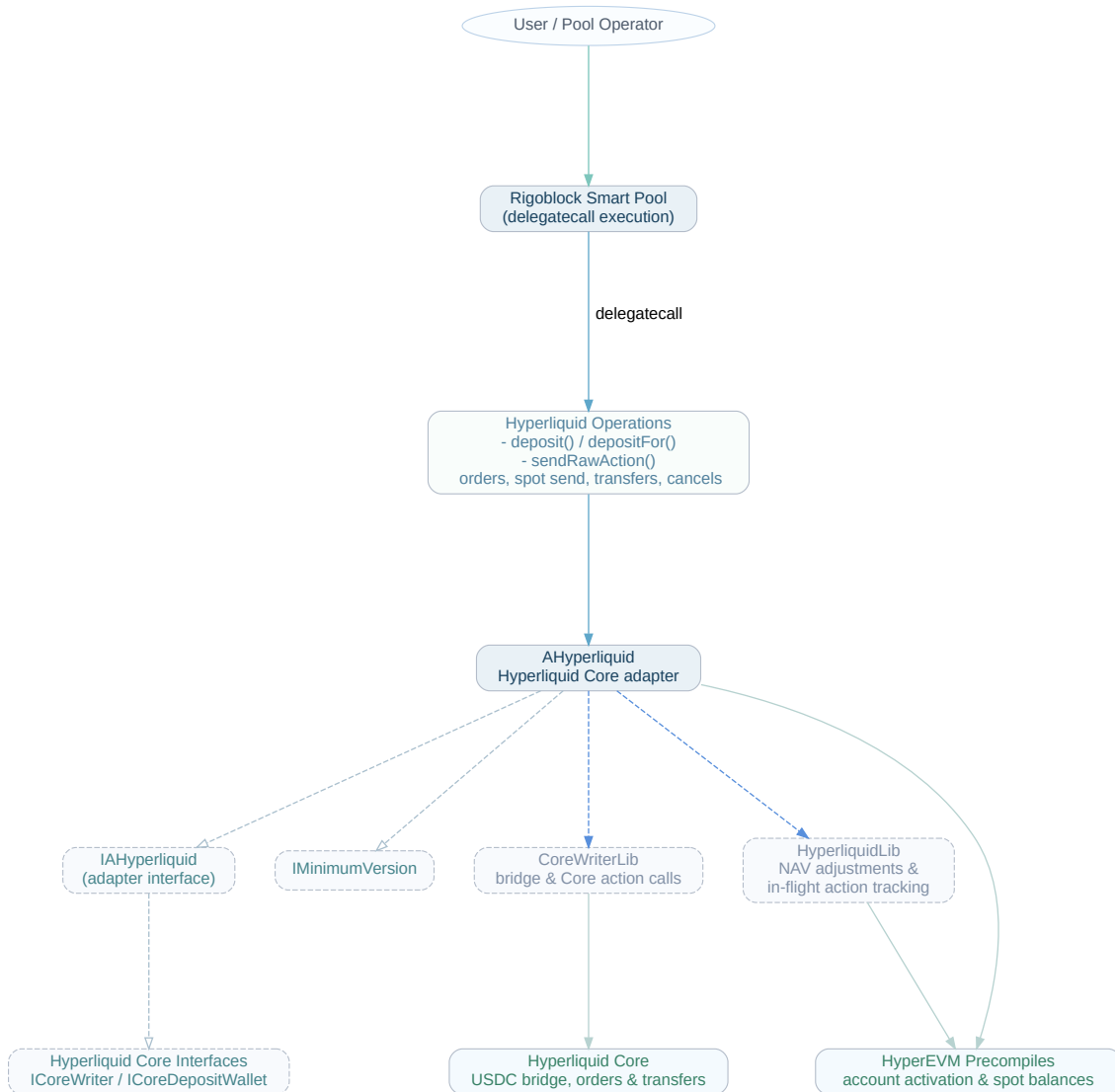
The Rigoblock Hyperliquid Adapter protocol provides integration between Rigoblock Smart Pools and Hyperliquid Core on HyperEVM. It enables Smart Pools to manage collateral, bridge USDC, and execute trading actions on Hyperliquid's perpetual and spot DEX infrastructure.

The protocol operates through `AHyperliquid`, a delegate-call adapter that handles deposits and forwards action payloads to Hyperliquid's core engine while tracking pool assets. It maintains dynamic Net Asset Value (NAV) calculations across spot and perpetual account balances using `HyperliquidLib`, which records in-flight actions to prevent arbitrage and balance discrepancies within the block execution window.

Main Entry Points

1. `deposit(uint256 amount, uint32 destinationDex)`: Bridges USDC from the Smart Pool to Hyperliquid Core to fund perpetual or spot trading accounts.
- Actors: Smart Pool Managers, Smart Pools.
2. `depositFor(address recipient, uint256 amount, uint32 destinationDex)`: Bridges USDC to Hyperliquid Core specifically ensuring the caller pool is the credited recipient.
- Actors: Smart Pool Managers, Smart Pools.
3. `sendRawAction(bytes calldata data)`: Dispatches trade actions such as placing limit orders, canceling orders by order ID or client order ID, transferring funds between spot and perp accounts, and bridging spot USDC.
- Actors: Smart Pool Managers, Smart Pools.

Code Diagram



✦ 1 of 10 Findings

📁 contracts/protocol/extensions/adapters/AHyperliquid.sol

Untracked spot-to-perp transfers permit overcommitted same-block spot-send accounting

• Medium Risk

`_transferUsdClass` does not reserve or otherwise record the reduction of Core spot USDC when `toPerp` is true. In the same block, `_spotSend` still validates against the stale `PrecompileLib.spotBalance` and only subtracts prior `pendingSpotSend` values. Thus, after moving spot USDC to perp, the pool can submit a spot send for up to the stale spot balance minus `_BRIDGE_GAS_RESERVE`; the adapter accepts it and records a negative in-flight NAV adjustment despite the previously submitted transfer having consumed the actual spot liquidity. Under delayed Core settlement, an overcommitted spot send that is rejected or not executed leaves a false withdrawal adjustment, allowing materially understated same-block NAV and NAV-dependent misaccounting.

Severity Note:

- `updateUnitaryValue()` uses `getHyperliquidBalances()` and incorporates the negative `recordAction` adjustment during the relevant pricing window, as stated in the finding but not shown in the supplied adapter.
- A deposit, redemption, share mint, or other economically consequential NAV-dependent operation can occur while the erroneous negative adjustment is live.
- The Core spot send can be rejected, deferred, or otherwise fail to consume the booked spot balance without synchronously reverting the adapter transaction, as specified in the validated attack path.

✦ 2 of 10 Findings

📁 contracts/protocol/extensions/adapters/AHyperliquid.sol

Core deposits omit the pre-call USDC approval required by the deposit wallet

• Medium Risk

`AHyperliquid._bridgeUsdcToCore` invokes `CoreWriterLib.bridgeUsdcToCoreFor(recipient, amount, destinationDex)` without first approving `HLConstants.coreDepositWallet()` to spend the pool's USDC. Its only approval-related code runs after that external bridge call and therefore cannot authorize it. Consequently, a normal pool with no sufficient pre-existing deposit-wallet allowance cannot fund its HyperCore account through `deposit` or `depositFor`; the bridge pull reverts before the adapter reaches its post-call allowance-reset logic. A separately established exact allowance may work around this, so the defect is the broken default adapter flow rather than an unconditional permanent failure under every allowance state.

✦ 3 of 10 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

Core-deposit flow lacks the required pre-call USDC approval

• Medium Risk

`_bridgeUsdcToCore` calls `CoreWriterLib.bridgeUsdcToCoreFor` before approving `HLCConstants.coreDepositWallet()` for `amount`. The only allowance operation is executed after the bridge call, so it cannot authorize an initial deposit-wallet `transferFrom`. As a result, a pool with a zero or insufficient pre-existing allowance cannot deposit USDC into HyperCore through this adapter until an allowance is separately provisioned. The adapter should approve the exact amount before calling the bridge and then perform its intended post-call allowance reset.

4 of 10
Findings

contracts/protocol/libraries/HyperliquidLib.sol

contracts/protocol/extensions/adapters/AHyperliquid.sol

Same-block-only HyperCore lag compensation causes stale NAV after deposits and withdrawals

• Medium Risk

`HyperliquidLib` only applies settlement-lag compensation in the exact block where the Hyperliquid action was submitted, even though the project context states that HyperCore state can lag HyperEVM writes by at least one block. The relevant logic is:

```
uint256 private constant _ACTION_BLOCK_WINDOW = 0;

function hasRecentAction() private view returns (bool) {
    uint256 lastBlock = StorageLib.hyperliquidData().lastActionBlock;
    return lastBlock != 0 && block.number <= lastBlock + _ACTION_BLOCK_WINDOW;
}

function getHyperliquidBalances(address account) internal view returns (AppTokenBalance[]
memory balances) {
    ...
    bool recentAction = hasRecentAction();
    if (recentAction) {
        totalUsdcValue += StorageLib.hyperliquidData().inFlightAmount;
    }
    ...
}
```

`AHyperliquid` relies on this mechanism to compensate for delayed Core settlement after USDC-moving actions:

```
function _bridgeUsdcToCore(address recipient, uint256 amount, uint32 destinationDex) private
{
    ...
    HyperliquidLib.recordAction(amount.toInt256());
}

function _spotSend(bytes calldata params) private {
    ...
    HyperliquidLib.recordAction(-SafeCast.toInt256(HLConversions.weiToEvm(token, amount)));
    CoreWriterLib.spotSend(destinationAddress, token, amount);
}
```

Because `_ACTION_BLOCK_WINDOW` is `0`, the in-flight delta disappears as soon as the chain advances to the next block. If HyperCore's precompile view is still stale in block `N+1`, `getHyperliquidBalances()` reads the old Core state without the compensating adjustment:

- after a Core deposit, pool-side EVM USDC has already been spent, but the Core balance may still omit the deposit, so NAV is understated;
- after a `SPOT_SEND`, the Core balance may still include the withdrawn USDC while the EVM side may already receive or later receive the bridged USDC, so NAV can be overstated or double-counted.

This is economically dangerous because the project context states that pool NAV is used for smart-pool accounting and cross-chain donation / transfer flows. Any share mint, burn, or other NAV-dependent operation executed in the stale block window can occur at the wrong price and transfer value between users.

Severity Note:

- Pool NAV is actually consumed by an executable mint, burn, donation, transfer, or equivalent operation during the stale block, rather than solely being displayed or used for non-executable reporting.
- A participant can benefit from an incorrect NAV-dependent operation, such as by minting or redeeming shares, and the relevant pool logic does not independently defer, snapshot, or validate HyperCore settlement.
- HyperCore can remain stale into block N+1 for the affected bridge/spot-send actions, as stated in the finding context.

5 of 10
Findingscontracts/protocol/extensions/adapters/AHyperliquid.sol
contracts/protocol/libraries/HyperliquidLib.sol**Marketable limit orders are treated as NAV-neutral even though same-block fills can change account value**

• Low Risk

`sendRawAction()` records a zero in-flight adjustment for every supported raw action except the explicit USDC bridge paths. For `LIMIT_ORDER_ACTION`, the adapter forwards arbitrary order parameters to Hyperliquid and then only executes `HyperliquidLib.recordAction(0)`. At the same time, `HyperliquidLib.getHyperliquidBalances()` derives NAV from `PrecompileLib.accountMarginSummary(...).accountValue` and spot USDC, and only compensates stale same-block state with the stored `inFlightAmount`.

This creates a mismatch for marketable limit orders. `_placeLimitOrder()` does not restrict `limitPx` or `encodedTif`, so the pool manager can submit an order that crosses the book and fills immediately. A same-block fill can change the Hyperliquid account's economic value through trading fees, spread paid, and immediate mark-to-market effects, but none of that is reflected in `inFlightAmount` because the action is recorded as zero-impact:

```
if (actionId == HLConstants.LIMIT_ORDER_ACTION) {\n  _placeLimitOrder(params);\n}\n...\nHyperliquidLib.re
```

Meanwhile, NAV continues to use the stale precompile view until HyperCore catches up:

```
int256 totalRawWei = int256(PrecompileLib.accountMarginSummary(HLConstants.DEFAULT_PERP_DEX, account).ac
```

As a result, after a same-block executable order, the pool can report pre-trade Hyperliquid NAV even though value has already moved on the exchange. Any pool logic that relies on same-block NAV freshness, such as share issuance/redemption or cross-chain accounting that reads NAV after `updateUnitaryValue()`, can operate on a materially stale price basis. The issue is especially acute because the adapter explicitly exposes raw order placement, and the order parameters permit immediately executable orders rather than only passive resting orders.

Severity Note:

- A pool function materially uses `getHyperliquidBalances()` or a derived unitary value for issuance, redemption, cross-chain accounting, or another value-transferring operation in the same block as `sendRawAction()`.
- A manager or colluding participant can cause such a NAV-dependent operation to execute during the one-block stale window and economically benefit from the pre-trade valuation.
- The relevant HyperCore precompile state actually lags the immediate fill as described; the supplied contracts are designed around this premise but do not independently prove the external settlement behavior.

✦ 6 of 10 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

Deprecated OpenZeppelin Function

• Low Risk

The adapter calls `usdc.safeApprove(depositWallet, 1);`, which relies on an OpenZeppelin approval helper that is considered deprecated in favor of newer allowance-setting patterns. In this contract, the approval is performed on the USDC token for the `depositWallet` address, so the flagged line is the exact place where the deprecated helper is used.

✦ 7 of 10 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

Large Numeric Literal

• Low Risk

The constant `_ASSET_ID_CORE_SPOT_BASE` is assigned the raw literal `10_000`. Although this value is valid, the numeric constant is written as a plain decimal literal instead of a more compact constant expression style, which makes the intent less obvious when reviewing asset identifiers in the adapter.

✦ 8 of 10 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

Literal Instead of Constant

• Low Risk

The same numeric slice length is hardcoded in `uint24 actionId = uint24(bytes3(data[1:4]));`. Because the code extracts exactly bytes `1` through `3` from `data` to derive `actionId`, the literal range is embedded directly at the call site instead of being centralized as a named constant or reused value, which makes the parsing format harder to audit and maintain.

✦ 9 of 10 Findings

contracts/protocol/extensions/adapters/AHyperliquid.sol

State Change Without Event

• Low Risk

The `depositFor(...)` flow updates contract state, but no event is emitted to record the change. As a result, off-chain consumers cannot rely on the event log to observe when this function mutates storage or other tracked state, and the state transition is only visible by reading contract storage directly.

✦ 10 of 10 Findings

📁 contracts/protocol/extensions/adapters/AHyperliquid.sol

Unused Import

• Low Risk

The import of `ICoreDepositWallet` is present in the adapter source, but the referenced type is not used anywhere in the code shown. This adds dead code to the compilation unit and suggests the dependency is no longer needed in the current implementation.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.